



✓ Working with Zuul API Gateway

- ✓ Zuul Server is an API Gateway Server.
- ✓ Zuul Server handles all the requests and performs the dynamic routing of microservice applications.
- ✓ Zuul Server is also known as Edge Server.
- ✓ Zuul is built to enable dynamic routing, monitoring, resiliency, and security.
- ✓ The Zuul Server dynamically routes the requests to the respective back-end application.

Working Steps:

- 1) Take the WorkSpace : **TopicDemos\Zuul-Demo**
- 2) Copy the following 3 apps from **Opt6-Using-Feign-Eureka**
 - a. MyEurekaServer
 - b. BookPriceMS
 - c. BookSearchMS
- 3) Copy the following 1 app1 from **Hystix-Demo**
 - a. BookStoreWeb
- 4) **Eureka Server– (Use from Previous Labs)**
- 5) Create the ApiGatewayServer – **MyApiGatewayServer** (Code Given Below)
- 6) Update the MicroService – **BookPriceMS** (Code Given Below)
- 7) Update the MicroService – **BookSearchMS** (Code Given Below)
- 8) Update the MicroService – **BookStoreMS** (Code Given Below)
- 9) Start **Eureka Server** (Port : 8761)
- 10) Start the **ApiGatewayServer** (Port : 5678)
- 11) Start 1 Instance of **BookPriceMS** (Port : 9000)
- 12) Start 1 Instance of **BookSearchMS** (Port : 8000)
- 13) Start 1 Instance of **BookStoreWeb** (Port : 5000)
- 14) Open the Browser and hit the following URL
<http://localhost:5000/jlcbook/105>
<http://localhost:5678/myjlc-book-store/jlcbook/105>
- 15) See the Console of **ApiGatewayServer** --



MyApiGatewayServer – Files Required

1. MyGatewayServerApplication.java	2. MyLogFilter.java
3. MySecurityFilter.java	4. application.properties
5. pom.xml	

1. MyGatewayServerApplication.java

```
package com.jlcindia.mygatewayserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
import org.springframework.cloud.netflix.zuul.EnableZuulProxy;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableEurekaClient
@EnableZuulProxy
public class MyGatewayServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyGatewayServerApplication.class, args);
    }
}
```

2. MyLogFilter.java

```
package com.jlcindia.mygatewayserver;

import javax.servlet.http.HttpServletRequest;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

import com.netflix.zuul.ZuulFilter;
import com.netflix.zuul.context.RequestContext;
import com.netflix.zuul.exception.ZuulException;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



@Component

```
public class MyLogFilter extends ZuulFilter{
```

```
    Logger logger=LoggerFactory.getLogger(this.getClass());
```

```
    @Override
```

```
    public Object run() throws ZuulException {
```

```
        logger.info("---1. MyLogFilter---run()");
```

```
        // Write your logic Here
```

```
        HttpServletRequest request=RequestContext.getCurrentContext().getRequest();
```

```
        String URI=request.getRequestURI();
```

```
        logger.info(" Request From => "+URI);
```

```
        return null;
```

```
    }
```

```
    @Override
```

```
    public String filterType() {
```

```
        return "pre";
```

```
    }
```

```
    @Override
```

```
    public int filterOrder() {
```

```
        return 1;
```

```
    }
```

```
    @Override
```

```
    public boolean shouldFilter() {
```

```
        return true;
```

```
    }
```

```
    }
```

3. MySecurityFilter.java

```
package com.jlcindia.mygatewayserver;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
import org.springframework.stereotype.Component;
```

```
import com.netflix.zuul.ZuulFilter;
```

```
import com.netflix.zuul.exception.ZuulException;
```

```
/*
```

```
 * @Author : Srinivas Dande
```

```
 * @company : Java Learning Center
```

```
 */
```



@Component

```
public class MySecurityFilter extends ZuulFilter{  
  
    Logger logger=LoggerFactory.getLogger(this.getClass());
```

@Override

```
public Object run() throws ZuulException {  
    logger.info("---2. MySecurityFilter---run()");  
    // Write your logic Here
```

```
    logger.info("Authetication Done Successfully");  
    return null;  
}
```

@Override

```
public String filterType() {  
    return "pre";  
}
```

@Override

```
public int filterOrder() {  
    return 2;  
}
```

@Override

```
public boolean shouldFilter() {  
    return true;  
}  
}
```

4. application.properties

```
spring.application.name=MyApiGatewayServer  
server.port=5678
```

```
zuul.routes.book-store.path=/myjlc-book-store/**  
zuul.routes.book-store.url=MyBookStoreWeb
```

```
zuul.routes.book-search.path=/myjlc-book-search/**  
zuul.routes.book-search.url=MyBookSearchMS
```

```
zuul.routes.book-price.path=/myjlc-book-price/**  
zuul.routes.book-price.url=MyBookPriceMS
```



```
zuul.routes.web-site.path=/mygoogle-site/**
zuul.routes.web-site.url=https://www.google.com

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

5. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.mygatewayserver</groupId>
<artifactId>MyApiGatewayServer</artifactId>
<version>1.0</version>
<name>MyApiGatewayServer</name>
<description> Zuul - MyApiGatewayServer</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-zuul</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

BookPriceMS – Files Required

1. BookPriceApplication.java	2. BookPriceController.java
3. BookPriceInfo.java	4. application.properties
5. pom.xml	

1. BookPriceApplication.java

```
package com.jlcindia.bookprice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



@SpringBootApplication

@EnableEurekaClient

```
public class BookPriceApplication {  
    public static void main(String args[]) {  
        SpringApplication.run(BookPriceApplication.class, args);  
    }  
}
```

2. BookPriceController.java

```
package com.jlcindia.bookprice;
```

```
import org.springframework.beans.factory.annotation.Value;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.PathVariable;  
import org.springframework.web.bind.annotation.RestController;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @company : Java Learning Center
```

```
*/
```

@RestController

```
public class BookPriceController {
```

@Value("\${server.port}")

String bookPriceServerPort;

@GetMapping("/bookPrice/{bookId}")

```
public BookPriceInfo getBookPrice(@PathVariable Integer bookId) {  
    System.out.println("---BookPriceController---getBookPrice()--: "+bookPriceServerPort);
```

```
    BookPriceInfo bookPriceInfo =
```

```
    new BookPriceInfo(bookId, 5000, 50,bookPriceServerPort);
```

```
    return bookPriceInfo;
```

```
}
```

```
}
```



3. BookPriceInfo.java

```
package com.jlcindia.bookprice;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class BookPriceInfo {
private Integer bookId;
private double price;
private double offer;
private String bookPriceServerPort;

//Constructors
//Setters and Getters

}
```

4. application.properties

```
spring.application.name=MyBookPriceMS

server.port=9000

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

5. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.bookprice</groupId>
<artifactId>BookPriceMS</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>

<name>MyBookPriceMS</name>
<url>https://www.jlcindia.com</url>
<description>This is BookPrice MS developed by Srinivas Dande</description>
```



```
<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
```



```
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```

BookSearchMS – Files Required

1. BookSearchApplication.java	2. BookSearchController.java
3. BookPriceProxy.java	4. BookPriceInfo.java
5. BookInfo.java	6. application.properties
7. pom.xml	

1. BookSearchApplication.java

```
package com.jlcindia.booksearch;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
import org.springframework.cloud.openfeign.EnableFeignClients;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@SpringBootApplication
@EnableEurekaClient
@EnableFeignClients
public class BookSearchApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookSearchApplication.class, args);
    }
}
```



2. BookSearchController.java

```
package com.jlcindia.booksearch;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class BookSearchController {

    @Value("${server.port}")
    String bookSearchServerPort;

    @Autowired
    BookPriceProxy bookPriceProxy;

    @GetMapping("/mybook/{bookId}")
    public BookInfo getBookById(@PathVariable Integer bookId) {
        System.out.println("---BookSearchController---getBookById()-----"+bookSearchServerPort);
        BookInfo bookInfo = new BookInfo();

        bookInfo.setBookId(bookId);
        bookInfo.setBookName("Master Spring MicroServices");
        bookInfo.setAuthor("Srinivas Dande");
        bookInfo.setCategory("Java");
        bookInfo.setPublications("JLC");

        // Need to Invoke BookPriceMS

        BookPriceInfo bookPriceInfo = bookPriceProxy.getBookPrice(bookId);
        bookInfo.setPrice(bookPriceInfo.getPrice());
        bookInfo.setOffer(bookPriceInfo.getOffer());
        bookInfo.setBookPriceServerPort(bookPriceInfo.getBookPriceServerPort());
        bookInfo.setBookSearchServerPort(bookSearchServerPort);

        return bookInfo;
    }
}
```



3. BookPriceInfo.java

```
package com.jlcindia.bookprice;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class BookPriceInfo {
    private Integer bookId;
    private double price;
    private double offer;
    private String bookPriceServerPort;

    //Constructors
    //Setters and Getters

}
```

4. BookInfo.java

```
package com.jlcindia.booksearch;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class BookInfo {
    private Integer bookId;
    private String bookName;
    private String author;
    private String publications;
    private String category;
    private double price;
    private double offer;
    private String bookPriceServerPort;
    private String bookSearchServerPort;

    //Constructors
    //Setters and Getters

}
```



5. BookPriceProxy.java

```
package com.jlcindia.booksearch;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@FeignClient( "MyApiGatewayServer")
public interface BookPriceProxy {

    @GetMapping("/myjlc-book-price/bookPrice/{bookId}")
    public BookPriceInfo getBookPrice(@PathVariable Integer bookId);
}
```

6. application.properties

```
spring.application.name=MyBookSearchMS

server.port=8000

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

7. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.booksearch</groupId>
<artifactId>BookSearchMS</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>

<name>MyBookSearchMS</name>
<url>https://www.jlcindia.com</url>
<description>This is BookSearch MS developed by Srinivas Dande</description>
```



```
<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
<relativePath />
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>
```



```
<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```

BookStoreWeb – Files Required

1. BookStoreApplication .java	2. BookStoreController.java
3. BookInfo.java	4. BookSearchProxy.java
5. application.properties	6. pom.xml

1. BookStoreApplication .java

```
package com.jlcindia.bookstore;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
import org.springframework.cloud.openfeign.EnableFeignClients;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@SpringBootApplication
@EnableFeignClients
@EnableEurekaClient
```



```
public class BookStoreApplication {  
    public static void main(String args[]) {  
        SpringApplication.run(BookStoreApplication.class, args);  
    }  
}
```

2. BookStoreController.java

```
package com.jlcindia.bookstore;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.beans.factory.annotation.Value;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.PathVariable;  
import org.springframework.web.bind.annotation.RestController;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
  
@RestController  
public class BookStoreController {  
  
    @Value("${server.port}")  
    String bookStoreServerPort;  
  
    @Autowired  
    BookSearchProxy bookSearchProxy;  
  
    @GetMapping("/jlcbook/{bookId}")  
    public BookInfo getBookById(@PathVariable Integer bookId) {  
        System.out.println("---BookStoreController---getBookById()-----"+bookStoreServerPort);  
  
        BookInfo bookInfo = bookSearchProxy.getBookById(bookId);  
  
        bookInfo.setBookStoreServerPort(bookStoreServerPort);  
  
        return bookInfo;  
    }  
}
```



3. BookInfo.java

```
package com.jlcindia.booksearch;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class BookInfo {
private Integer bookId;
private String bookName;
private String author;
private String publications;
private String category;
private double price;
private double offer;
private String bookPriceServerPort;
private String bookSearchServerPort;
private String bookStoreServerPort;

//Constructors
//Setters and Getters

}
```

4. BookSearchProxy.java

```
package com.jlcindia.bookstore;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@FeignClient(name="MyApiGatewayServer")
public interface BookSearchProxy {

    @GetMapping("/myjlc-book-search/mybook/{bookId}")
    public BookInfo getBookById(@PathVariable Integer bookId) ;

}
```



5. application.properties

spring.application.name=**MyBookStoreWeb**

server.port=**5000**

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

eureka.client.healthcheck.enabled=true

logging.level.root=INFO

logging.pattern.console=%-5level %logger{36} - %msg%n

6. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.bookstore</groupId>
<artifactId>BookStoreWeb</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>

<name>MyBookStoreWeb</name>
<url>https://www.jlcindia.com</url>
<description>This is BookStoreWeb developed by Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>
</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```